TAR System Description Paper Template

Author1, Author2, Author3

University of Zagreb, Faculty of Electrical Engineering and Computing
Unska 3, 10000 Zagreb, Croatia
autor1@xxx.hr, {autor2,autor3}@zz.com

Abstract

This document provides the instructions on formatting the TAR system description paper in LaTeX. This is where you write the abstract (i.e., summary) of the work you carried out within the project. The abstract is a paragraph of text ranging between 70 and 150 words.

1. Introduction

Text for the introduction goes here.

2. Related work

Text for the related work goes here.

3. Methodology

In this section, we describe the established framework for evaluating the effectiveness and robustness of pairing Negative Preference Optimisation (NPO) with Low-Rank Adaptation (LoRA) for unlearning in large language models. We also define the evaluation framework for the adversarial jailbreak attacks. We detail the datasets, models, and evaluation metrics used to assess the performance of this approach.

3.1. Tasks and dataset

The primary dataset used in our experiments is part of the *LUME: LLM Unlearning with Multitask Evaluations* benchmark. The dataset includes documents that are categorised into three scenarios:

(I) **Synthetic creative documents** These documents are synthetic fictional stories in various genres.

(II) **PII-containing synthetic biographies** These are synthetic biographies of public figures which contain personally identifiable information (PII), including names, home addresses, and Social Security numbers.

(III) **Real documents from the pretraining corpus** These are real documents that were included in the pretraining corpus of the language model.

Each subtask has two main data splits: *Forget* and *Retain*. The *Forget* split consists of documents that the model should unlearn, while the *Retain* split includes documents that should be retained in the model's memory. The number of examples can be found in Table 1.

Table 1: Distribution of documents in the dataset.

Document	Forget	Retain	Total
Synthetic Creative Documents	166	206	372
Synthetic Biographies	642	612	1254
Real Documents	304	318	622
Total	1112	1136	2248

3.2. Model

We conduct our experiments using the *OLMo-7B-0724-Instruct-hf* model, which has been explicitly fine-tuned to memorise the documents from the LUME benchmark dataset.

Unlearning the above-mentioned model is achieved through Negative Preference Optimisation (NPO). Since this method is computationally heavy, we use it in combination with Low-Rank Adaptation (LoRA) to decrease the number of trainable parameters, making the unlearning process more efficient. All experiments were run on a single NVIDIA A100 40GB GPU.

Given a prompt x, the training objective of our approach is to minimise the likelihood of generating forgotten documents y_f, compared to a fixed reference model π_{ref}. Formally, the training objective can be expressed as follows:

$$\mathcal{L}_{NPO}(\theta) = \frac{2}{\beta} \log \left(1 + \left(\frac{\pi_\theta(y_f|x)}{\pi_{ref}(y_f|x)} \right)^\beta \right),$$

where π_θ is the model being trained, π_{ref} is the reference model, and β is a hyperparameter that controls the strength of the preference. Additionally, to maintain the model's general capabilities, we include an additional cross-entropy loss term that encourages the model to keep the knowledge of the retained documents y_r. The final loss function is:

$$\mathcal{L}(\theta) = \mathcal{L}_{NPO}(\theta) + \alpha \cdot \mathcal{L}_{CE}(\theta),$$

where $\mathcal{L}_{CE}(\theta)$ is the cross-entropy loss computed on the retained documents, and α is a hyperparameter that aids in balancing unlearning and knowledge retention.

To further strengthen our jailbreak evaluation comparisons, we implemented Activation Steering method, which was proposed in (Stolfo et al., 2024). Activation steering identifies a desired direction in the model's latent space and surpresses or enhances it at inference time. Given a set of N pairs (x_f, x_r), where x_f represents a forget-set prompt and x_r represents a retain-set prompt, we extract hidden-state activations h_f and h_r at decoder layer $\ell = 20$ by averaging over the token dimension. We compute the steering vector

as the mean difference:

$$\mathbf{v} = \frac{1}{N}\sum_{i=1}^{N}\left(h_f^{(i)} - h_r^{(i)}\right), \quad \hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|},$$

producing a unit-normalised steering vector $\hat{\mathbf{v}} \in \mathbb{R}^d$. During inference, we apply a forward hook on layer ℓ that modifies the forward pass activations at each token position:

$$h_\ell^{\text{out}} \leftarrow h_\ell^{\text{out}} + \alpha \cdot \hat{\mathbf{v}},$$

where $\alpha < 0$ is the steering strength. Setting α to a negative value subtracts the memorisation direction, which suppresses information recall without modifying the model weights. In our experiments, we use $N = 64$ contrast pairs and $\alpha = -5.0$, selected by sweeping over $\alpha \in \{-2.0, -5.0, -8.0, -12.0, -20.0\}$ to balance unlearning effectiveness and output quality. The steering vector comes from the baseline model fine-tuned to memorise the dataset, before unlearning, and it remains fixed throughout training and evaluation.

3.3. Adversarial Red-Teaming

To test the robustness of NPO-LoRA against adversarial attacks, we set up an adversarial red-teaming procedure. We used *gpt-4o-mini* to rewrite the queries from the *Forget* split into seven different adversarial prompts:

- **Paraphrase** - changing the wording of the original query while keeping its meaning.
- **Prefix Injection** - prepends bypass instructions to the original query.
- **Roleplay** - presenting the query as a roleplay scenario.
- **Translation** - translating the original query into another language.
- **Code Context** - embedding the original query within a code snippet.
- **Neighbour** - focusing on nearby documents without referencing the original query.
- **Logical Chain** - using multi-step reasoning that leads back to the original query.

3.4. Evaluation Metrics

Our evaluation framework relies on the competition metrics, combining three main dimensions through an arithmetic mean to produce a final submission score. First, we calculate task-specific rates across the *Forget* and *Retain* splits. For sentence completion tasks, a ROUGE-L regurgitation score is used, while for question-answering tasks, we use an Exact Match (EM) score. The metrics for the forget split are inverted (1 − value). In total, 12 scores across the three tasks are then combined via the harmonic mean into a single score that represents the overall performance of the unlearning approach. Second, a Membership Inference Attack (MIA) score is calculated over member and non-member documents to assess the model's vulnerability to privacy attacks. Third, general capabilities are evaluated using the MMLU benchmark, which consists of 57 multiple-choice tasks, representing the overall performance of the model on a wide range of general knowledge and reasoning tasks.

4. Results

We assesss the performance of NPO-LoRA by tracking metrics across various LoRA adapter capacities and NPO hyperparameters. The results are displayed in Table 2 in a split format with three column groups: hyperparameters, base metrics, and aggregate scores. The *Hyperparameters* column shows the LoRA rank (r), LoRA scaling factor (α_{LoRA}), NPO preference strength (β_{NPO}), NPO knowledge retention weight (α_{NPO}), learning rate (LR), and the number of training epochs (Ep.). The *Base Metrics* column reports the performance on the *Forget* and *Retain* splits, with arrows (↓, ↑) indicating the desired direction of improvement. Finally, the *Aggregate* section presents the Membership Inference Attack (MIA) score and the overall final score (Agg).

The **Baseline** model represents the original model without any unlearning applied. This model achieves a maximum score of 1.000 across all base metrics and a MIA score of 0.0, resulting in an (lowest) overall aggregate score of 0.168, which serves as a reference point for evaluating the effectiveness of different NPO-LoRA configurations.

Configurations with low to mid-capacity LoRA adapters ($r = 8, 16$) showed limited performance, with the best configuration (Run 3) achieving an aggregate score of 0.230. Increasing the LoRA adapter capacity to $r = 32$ (Runs 5-8) led to significant improvements in the aggregate score, with the best configuration (Run 6) reaching an aggregate score of 0.318. This configuration also had the lowest F-Reg (0.720) and F-Know (0.553) scores. Run 7, which had a slightly higher NPO knowledge retention weight, achieved a marginally lower aggregate score of 0.301, while maintaining higher R-Reg (0.968) and R-Know (0.929) scores, though with slightly higher F-Reg (0.779) and F-Know (0.583) scores.

Across all configurations evaluated, including the baseline, the MIA score remained constant (0.0).

5. Discussion

Text for the discussion goes here.

6. Conclusion

Conclusion is the last enumerated section of the paper. It should not exceed half of a column and is typically split into 2–3 paragraphs. No new information should be presented in the conclusion; this section only summarizes and concludes the paper.

Acknowledgements

We would like to thank the University Computing Centre (SRCE) in Zagreb for providing the computational resources on their Advanced Computing platform used in this work.

References

Alessandro Stolfo, Vidhisha Balachandran, Safoora Yousefi, Eric Horvitz, and Besmira Nushi. 2024.

Table 2: Hyperparameter ablation study including the **Baseline** model.

	Hyperparameters						Base Metrics				Aggregate	
Run	r	α_{LoRA}	β_{NPO}	α_{NPO}	LR	Ep.	F-Reg ↓	F-Know ↓	R-Reg ↑	R-Know ↑	MIA ↑	Agg ↑
Baseline	-	-	-	-	-	-	1.000	1.000	1.000	1.000	0.0	0.168
Low / Mid Capacity Adapters ($r = 8, 16$)												
1	8	16	0.05	1.0	1e-5	2	0.953	0.960	0.960	0.969	0.0	0.169
2	16	32	0.10	1.5	3e-5	1	0.933	0.809	0.948	0.872	0.0	0.223
3	16	32	0.15	1.2	5e-5	2	0.925	0.819	0.982	0.962	0.0	0.230
4	16	32	1.00	1.0	5e-5	2	0.983	0.931	0.992	0.991	0.0	0.185
High Capacity Adapters ($r = 32$)												
5	32	64	0.08	0.5	3e-5	2	0.852	0.645	0.962	0.933	0.0	0.270
6	**32**	**64**	**0.10**	**0.1**	**1e-4**	**3**	**0.720**	**0.553**	0.885	0.848	0.0	**0.318**
7	**32**	**64**	**0.10**	**0.2**	**1e-4**	**3**	0.779	0.583	**0.968**	**0.929**	0.0	0.301
8	32	64	0.30	1.0	1e-4	4	0.913	0.797	0.989	0.986	0.0	0.232

Table 3: Attack Success Rate (ASR) summary across models.

	ASR by Model		
Attack	**Baseline**	**Run 6**	**Steering**
Prefix Injection	85.4%	78.2%	0.2%
Roleplay	78.5%	68.1%	0.0%
Paraphrase	73.7%	61.2%	1.8%
Translation	68.0%	60.0%	5.7%
Logical Chain	59.2%	48.9%	0.0%
Neighbour	45.0%	38.7%	0.8%
Code Context	1.1%	1.1%	0.0%

Improving instruction-following in language models through activation steering. *arXiv preprint arXiv:2410.12877*.